

Лабораторная работа № 14

**Программирование в командном процессоре ОС UNIX.
Ветвления и циклы (продолжение)**

Пириев Керем

Содержание

1	Цель работы	5
2	Задание	6
3	Выполнение лабораторной работы	7
3.1	1. Создание рабочего каталога и скрипт semaphore.sh	7
3.2	2. Реализация команды man (my_man.sh)	8
3.3	3. Генератор случайных букв (random_letters.sh)	9
4	Вывод	11
5	Контрольные вопросы	12
5.1	1. Найдите синтаксическую ошибку в следующей строке: while [$\backslash$ \$1 != "exit"]	12
5.2	2. Как объединить (конкатенация) несколько строк в одну?	12
5.3	3. Найдите информацию об утилите seq. Какими иными способами можно реализовать её функционал при программировании на bash?	12
5.4	4. Какой результат даст вычисление выражения $\$((10/3))$?	13
5.5	5. Укажите кратко основные отличия командной оболочки zsh от bash.	13
5.6	6. Проверьте, верен ли синтаксис данной конструкции: for ((\$a=1\$; a <= LIMIT; a++))	13
5.7	7. Сравните язык bash с какими-либо языками программирования. Какие преимущества у bash по сравнению с ними? Какие недостатки?	13
	Список литературы	15

Список иллюстраций

3.1	Скрипт semaphore.sh в редакторе nano	7
3.2	Запуск semaphore.sh	8
3.3	Скрипт my_man.sh для эмуляции man	8
3.4	Тестирование my_man.sh на примере ls	9
3.5	Скрипт random_letters.sh в nano	9
3.6	Запуск random_letters.sh и результат генерации	10

Список таблиц

1 Цель работы

Изучить основы программирования в оболочке ОС UNIX. Научиться писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

2 Задание

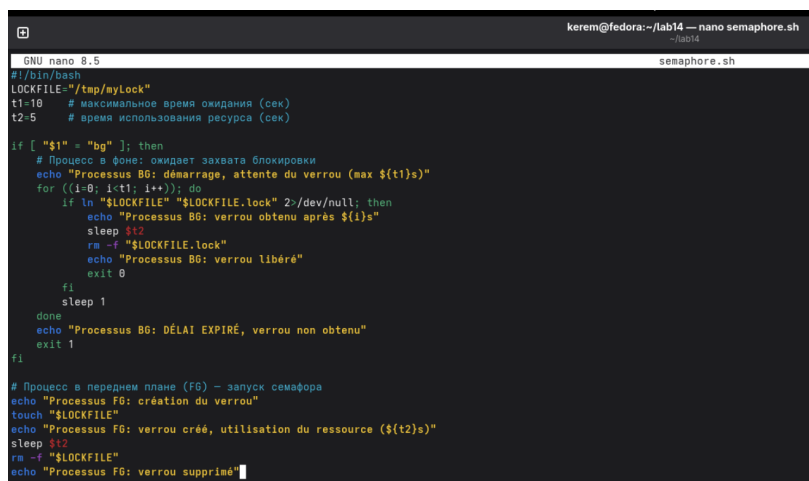
1. Создать рабочий каталог `~/lab14` для размещения всех скриптов.
2. Разработать скрипт `semaphore.sh` для демонстрации взаимодействия и синхронизации процессов с помощью файловых блокировок.
3. Разработать скрипт `my_man.sh` для эмуляции работы команды `man` (поиск и отображение страниц руководства, включая сжатые файлы).
4. Разработать скрипт `random_letters.sh` для генерации случайных строк заданной длины из букв латинского алфавита с помощью переменной `$RANDOM`.
5. Дать ответы на контрольные вопросы.

3 Выполнение лабораторной работы

3.1 1. Создание рабочего каталога и скрипт

semaphore.sh

Создан каталог `~/lab14`. Разработан скрипт `semaphore.sh`, демонстрирующий ожидание ресурса и управление блокировками (`/tmp/myLock`) между фоновыми и переднеплановыми процессами (рис. Рисунок 3.1).

A screenshot of a terminal window showing the nano text editor editing a file named semaphore.sh. The script contains logic for creating and managing a semaphore lock. It defines variables for lock file path, timeout, and resource usage time. It includes a function for background processes to wait for and acquire the lock, and another for foreground processes to create, use, and remove the lock. Comments in Russian explain the steps: creating the lock, waiting for it, acquiring it, using it, and removing it. The script uses standard shell constructs like if, for, sleep, touch, rm, and echo for output.

```
GNU nano 8.5 semaphore.sh
#!/bin/bash
LOCKFILE="/tmp/myLock"
t1=10 # максимальное время ожидания (сек)
t2=5 # время использования ресурса (сек)

if [ "$1" = "bg" ]; then
    # Процесс в фоне: ожидает захвата блокировки
    echo "Processus BG: démarrage, attente du verrou (max ${t1}s)"
    for ((i=0; i<t1; i++)); do
        if ln "$LOCKFILE" "$LOCKFILE.lock" 2>/dev/null; then
            echo "Processus BG: verrou obtenu après ${i}s"
            sleep $t2
            rm -f "$LOCKFILE.lock"
            echo "Processus BG: verrou libéré"
            exit 0
        fi
        sleep 1
    done
    echo "Processus BG: DÉLAI EXPIRÉ, verrou non obtenu"
    exit 1
fi

# Процесс в переднем плане (FG) - запуск семафора
echo "Processus FG: création du verrou"
touch "$LOCKFILE"
echo "Processus FG: verrou créé, utilisation du ressource (${t2}s)"
sleep $t2
rm -f "$LOCKFILE"
echo "Processus FG: verrou supprimé"
```

Рисунок 3.1: Скрипт semaphore.sh в редакторе nano

Проверено выполнение скрипта в терминале, отработал механизм создания и удаления блокировки (рис. Рисунок 3.2).

```

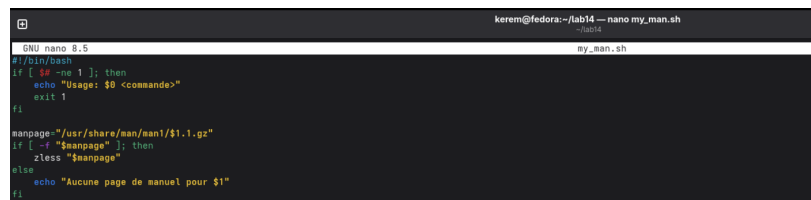
kerem@fedora:~/lab14$ cd ~/lab14
kerem@fedora:~/lab14$ chmod +x semaphore.sh
kerem@fedora:~/lab14$ ./semaphore.sh
Processus FG: création du verrou
Processus FG: verrou créé, utilisation du ressource (5s)
Processus FG: verrou supprimé
kerem@fedora:~/lab14$

```

Рисунок 3.2: Запуск semaphore.sh

3.2 2. Реализация команды man (my_man.sh)

Создан скрипт my_man.sh, который осуществляет поиск и постраничный вывод страниц руководства из каталога /usr/share/man/man1/, поддерживая работу со сжатыми файлами .gz (рис. Рисунок 3.3).



```

kerem@fedora:~/lab14 - nano my_man.sh
GNU nano 8.5 my_man.sh
#!/bin/bash
if [ $# -ne 1 ]; then
    echo "Usage: $0 <command>"
    exit 1
fi

manpage="/usr/share/man/man1/${1}.1.gz"
if [ -f "$manpage" ]; then
    zless "$manpage"
else
    echo "Aucune page de manuel pour $1"
fi

```

Рисунок 3.3: Скрипт my_man.sh для эмуляции man

Выполнено тестирование скрипта для команды ls (рис. Рисунок 3.4).

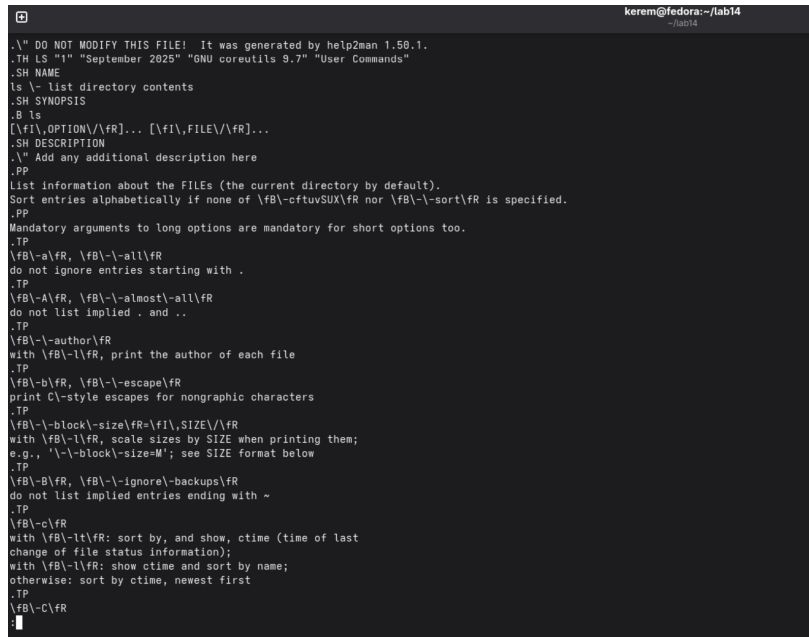


Рисунок 3.4: Тестирование `my_man.sh` на примере `ls`

3.3 3. Генератор случайных букв (random_letters.sh)

Создан скрипт `random_letters.sh` для генерации случайной последовательности символов указанной длины с использованием `$RANDOM` и подстановки строк (рис. Рисунок 3.5).

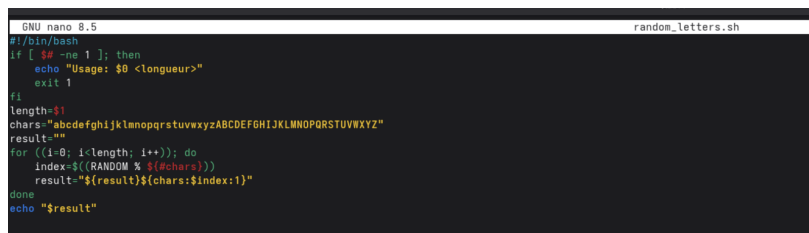


Рисунок 3.5: Скрипт random_letters.sh в nano

Проведено тестирование генерации строки длиной 10 символов (рис. Рисунок 3.6).

```
kerem@fedora:~/lab14$ ./my_man.sh ls
kerem@fedora:~/lab14$ nano random_letters.sh
kerem@fedora:~/lab14$ chmod +x random_letters.sh
kerem@fedora:~/lab14$ ./random_letters.sh 10
bAekrbXifB
kerem@fedora:~/lab14$
```

Рисунок 3.6: Запуск random_letters.sh и результат генерации

4 Вывод

В ходе работы были разработаны три скрипта: 1. Семафор (`semaphore.sh`) — демонстрация ожидания ресурса между двумя процессами (синхронизация с использованием таймаута). 2. Собственная команда `man` (`my_man.sh`) — поиск и отображение страниц руководства, включая сжатые (`.gz`). 3. Генератор случайных строк (`random_letters.sh`) — использование переменной `$RANDOM` для создания буквенной последовательности заданной длины. Получены навыки работы с блокировками, обработкой аргументов командной строки, чтением системных каталогов (`/usr/share/man`), а также генерацией случайных данных в `bash`.

5 Контрольные вопросы

5.1 1. Найдите синтаксическую ошибку в следующей строке: **while [\ \$1 != "exit"]**

Ошибка: не хватает пробелов внутри квадратных скобок, а также переменная \$1 должна быть взята в кавычки. Правильно: `while [ "$1" != "exit" ]`.

5.2 2. Как объединить (конкатенация) несколько строк в одну?

Использовать простое присваивание: `new="$str1$str2"` или с фигурными скобками: `new="{str1}${str2}"`.

5.3 3. Найдите информацию об утилите `seq`. Какими иными способами можно реализовать её функционал при программировании на `bash`?

`seq` — утилита для генерации последовательности чисел. Альтернативы в `bash`: цикл `for` в стиле Си (`for ((i=1; i<=N; i++))`) или конструкция `{1..N}`.

5.4 4. Какой результат даст вычисление выражения $\$((10/3))$?

Результат: 3 (целочисленное деление, остаток отбрасывается).

5.5 5. Укажите кратко основные отличия командной оболочки zsh от bash.

Zsh обладает более мощным автодополнением, встроенной поддержкой массивов (индексация с 1 по умолчанию), загружаемыми модулями и расширенной экосистемой плагинов (Oh My Zsh), в то время как bash строже следует стандарту POSIX.

5.6 6. Проверьте, верен ли синтаксис данной конструкции: `for (( $a=1$; a <= LIMIT; a++ ))`

Ошибка: лишние знаки доллара и некорректная расстановка символов. Правильная форма: `for (( a=1; a <= LIMIT; a++ ))`.

5.7 7. Сравните язык bash с какими-либо языками программирования. Какие преимущества у bash по сравнению с ними? Какие недостатки?

- Преимущества bash: идеален для автоматизации задач администрирования, быстрой работы с файлами, процессами и конвейерами команд.

- Недостатки: медленный для сложных вычислений, отсутствие прямой поддержки чисел с плавающей точкой, слабая типизация.

Список литературы

1. *Newham C.* Learning the bash Shell: Unix Shell Programming. — O'Reilly Media, 2005. — 354 p. — (In a Nutshell). — ISBN 0596009658. — URL: <http://www.amazon.com/Learning-bash-Shell-Programming-Nutshell/dp/0596009658>.
2. *Robbins A.* Bash Pocket Reference. — O'Reilly Media, 2016. — 156 p. — ISBN 978-1491941591.
3. *Zarrelli G.* Mastering Bash. — Packt Publishing, 2017. — 502 p. — ISBN 9781784396879.
4. *Таненбаум Э., Бос Х.* Современные операционные системы.. — 4-е изд. — СПб. : Питер, 2015.. — 1120 с. — (Классика Computer Science).